DESARROLLO DE APLICACIONES CON OBJETOS · 1º PARCIAL · TEMA 2

Objetos, herencia, polimorfismo, excepciones y testing

Los ejercicios del repositorio de la cátedra con el enunciado tal cual, la solución probada y la salida de la consola

12 ejercicios | Soluciones de la cátedra corregidas | Testing con pytest | Código para descargar

Cómo usar esta guía

Los ejercicios del **tema 2** del repositorio de la cátedra (2026-guias), en el mismo orden: objetos, herencia y polimorfismo, excepciones, testing y el ejercicio integrador. Cada uno tiene el **enunciado tal cual**, qué practica, la solución (una clase por archivo, como en clase) y la **salida real**. Donde la solución de la cátedra tenía errores se corrigieron y se explica qué se cambió.

Para correrlos: bajá el ZIP, abrí la carpeta del ejercicio en VS Code y ejecutá `principal.py` (o `python -m pytest` donde hay tests). Probado con Python 3.11.

Los **modelos de parcial** (La inmobiliaria, La fábrica y El supermercado) están resueltos en la **Guía del 1º parcial**.

	Ejercicio	Tema	Qué practica
01	**Personas**	Objetos	Definir una clase con constructor y `__str__`, crear objetos y comparar sus atributos sin guardarlos en una lista.
02	**Personas (desde archivo)**	Objetos	Leer un CSV, crear un objeto por línea y guardarlos en un diccionario indexado por documento; contar y agrupar.
03	**Veterinaria**	Objetos	**Composición**: el cliente *tiene una* mascota. Propiedades (`@property`) y recorrer una colección de objetos.
04	**Autos**	Objetos	Guardar el estado dentro del objeto y que los métodos apliquen las reglas (consumo, capacidad y reserva).
05	**Ascensor**	Objetos	Métodos que validan el estado y lo informan con el valor que devuelven (`True` / `False`, cantidad o `-1`). Trae **tests de pytest**.
06	**Partido de fútbol**	Objetos	Asociación entre objetos (el partido referencia a dos equipos) y atributos calculados con `@property`.
07	**Cuentas bancarias**	Herencia	**Herencia** y **clase abstracta**: cada tipo de cuenta implementa `extraer` a su manera (polimorfismo).
08	**Plantel de empleados**	Polimorfismo	**Polimorfismo** con datos de un archivo: cada empleado calcula su sueldo (`neto`) y la empresa los suma sin preguntar el tipo.
09	**Hospital**	Polimorfismo	Una colección con objetos de distintas clases (atenciones médicas y de farmacia) que responden al mismo mensaje.
10	**Transporte de cargas (integrador)**	Integrador	Todo junto: herencia, clase abstracta, composición, estados del camión, excepciones y lectura de CSV.
11	**Ejemplos de excepciones**	Excepciones	`try`, `except`, `else`, `finally` y cómo leer el error
12	**Testing con pytest**	Testing	Escribir pruebas con `assert`, correrlas y leer una prueba que falla

01 Personas

Objetos | Solución propuesta (la cátedra no trae)

Enunciado (tal cual)

Programar una clase Persona con atributos suficientes para almacenar documento, nombre, apellido y edad de una persona.

Crear un programa que permita ingresar los datos de algunas personas por teclado y con esos datos cree instancias sin guardarlas en una estructura de datos y muestre por pantalla el estado de las mismas.

Al finalizar la carga el programa debe mostrar los datos de la persona de menor edad.

Qué practica

Definir una clase con constructor y `__str__`, crear objetos y comparar sus atributos sin guardarlos en una lista.

persona.py

```
class Persona:
    def __init__(self, documento, nombre, apellido, edad):
        self.documento = documento
        self.nombre = nombre
        self.apellido = apellido
        self.edad = edad

    def __str__(self):
        return f"{self.documento} - {self.apellido}, {self.nombre} ({self.edad} años)"
```

principal.py

```python
from persona import Persona


def main():
    menor = None          # solo se recuerda la de menor edad: no se guardan en una estructura
    documento = int(input("Documento (0 para terminar): "))
    while documento != 0:
        nombre = input("Nombre: ")
        apellido = input("Apellido: ")
        edad = int(input("Edad: "))
        persona = Persona(documento, nombre, apellido, edad)
        print("Cargada:", persona)
        if menor is None or persona.edad < menor.edad:
            menor = persona
        documento = int(input("Documento (0 para terminar): "))

    if menor is None:
        print("No se cargó ninguna persona")
    else:
        print("La persona de menor edad es:", menor)


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Documento (0 para terminar): 30111222
Nombre: Ana
Apellido: López
Edad: 25
Cargada: 30111222 - López, Ana (25 años)
Documento (0 para terminar): 28999888
Nombre: Juan
Apellido: Pérez
Edad: 31
Cargada: 28999888 - Pérez, Juan (31 años)
Documento (0 para terminar): 40123456
Nombre: Sofía
Apellido: Gómez
Edad: 19
Cargada: 40123456 - Gómez, Sofía (19 años)
Documento (0 para terminar): 0
La persona de menor edad es: 40123456 - Gómez, Sofía (19 años)
```

02 Personas (desde archivo)

Objetos | Solución propuesta (la cátedra no trae)

Enunciado (tal cual)

Programar una clase Persona con atributos suficientes para almacenar documento, nombre, apellido y edad de una persona. Crear un programa que lea el archivo personas.csv, cree una instancia de Persona por cada línea del archivo y guarde todas las instancias en un diccionario indexado por documento.

A continuación, desde el diccionario informe:

- Cantidad de personas cargadas
- Cantidad de personas mayores de edad
- Listado de nombres y apellidos de aquellas personas cuyo apellido empiece en vocal
- Cantidad de personas por cada apellido

Qué practica

Leer un CSV, crear un objeto por línea y guardarlos en un diccionario indexado por documento; contar y agrupar.

Para tener en cuenta

- La clase `Persona` es la misma del ejercicio 1.
- El archivo `personas.csv` viene en el práctico de conjuntos y diccionarios (también está en el ZIP de este ejercicio).

principal.py

```python
from persona import Persona

VOCALES = "AEIOUÁÉÍÓÚ"


def leer_personas(nombre_archivo):
    """Devuelve un diccionario {documento: Persona} con todas las líneas del archivo."""
    personas = {}
    with open(nombre_archivo, encoding="utf-8") as archivo:
        for linea in archivo:
            linea = linea.strip()
            if linea:
                documento, nombre, apellido, edad = linea.split(",")
                personas[int(documento)] = Persona(int(documento), nombre, apellido, int(edad))
    return personas


def main():
    personas = leer_personas("personas.csv")

    print("Cantidad de personas cargadas:", len(personas))

    mayores = len([p for p in personas.values() if p.edad >= 18])
    print("Cantidad de mayores de edad:", mayores)

    print("Apellido que empieza con vocal:")
    for p in personas.values():
        if p.apellido[0] in VOCALES:
            print(f"   {p.nombre} {p.apellido}")

    por_apellido = {}
    for p in personas.values():
        por_apellido[p.apellido] = por_apellido.get(p.apellido, 0) + 1
    print("Cantidad de personas por apellido:")
    for apellido in sorted(por_apellido):
        print(f"   {apellido}: {por_apellido[apellido]}")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Cantidad de personas cargadas: 2000
Cantidad de mayores de edad: 1815
Apellido que empieza con vocal:
   IVAN ALVAREZ
   MARIA CARMEN AGUILAR
   MARC ORTIZ
   JUAN CARLOS ORTIZ
   TOMAS ORTEGA
   MIGUEL AGUILAR
   ANA ORTIZ
   JOSE ANGEL ALONSO
   CAROLINA ESTEBAN
   MOHAMED ALONSO
   IGNACIO AGUILAR
   MARGARITA ALVAREZ
   JESUS IGLESIAS
   ROSARIO IBAÑEZ
   MARIANO IGLESIAS
   ANDREA ORTIZ
   ROBERTO ALONSO
   JUAN LUIS ORTIZ
   ANDRES IGLESIAS
   LUISA AGUILAR
   MARIA LUISA ALVAREZ
   JUAN ANTONIO AGUILAR
   RUBEN ALVAREZ
… (260 renglones más)
   VARGAS: 19
   VAZQUEZ: 21
   VEGA: 16
   VELASCO: 20
   VICENTE: 24
   VIDAL: 21
```

03 Veterinaria

Objetos | Solución de la cátedra, corregida

Enunciado (tal cual)

Una Veterinaria encargada del cuidado de mascotas ha solicitado la realización de un software para obtener resultados sobre sus clientes y sus respectivas mascotas. (Suponer que cada cliente posee una sola mascota)

Se sabe que cada Cliente de esta veterinaria tiene: un numero de cliente, un nombre, una antigüedad (hace cuántos años que son clientes de la veterinaria) y una Mascota.

De la Mascota se dispone los siguientes datos: el nombre y la edad.

El software requiere cargar por teclado los datos de varios clientes y almacenarlos en alguna colección adecuada.

Se pide:

- Mostrar la cantidad de clientes.
- Mostrar el promedio de edad de las mascotas.
- Informar cuántos clientes tienen una antigüedad mayor igual a 5 años.
- Listado de los datos de todos los clientes cuya mascota tiene más de 5 años de edad.

Qué practica

Composición: el cliente *tiene una* mascota. Propiedades (`@property`) y recorrer una colección de objetos.

Qué se corrigió de la solución de la cátedra

- El setter de `antiguedad` modificaba el **número** de cliente (`self._numero = valor`).
- El enunciado pide antigüedad **mayor o igual** a 5; la solución contaba solo mayor a 5.

mascota.py

```python
class Mascota:
    def __init__(self, nombre, edad):
        self._nombre = nombre
        self._edad = edad

    @property
    def nombre(self):
        return self._nombre

    @property
    def edad(self):
        return self._edad

    @edad.setter
    def edad(self, valor):
        self._edad = valor

    def __str__(self):
        return f"{self._nombre} ({self._edad} años)"
```

cliente.py

```
class Cliente:
    def __init__(self, numero, nombre, antiguedad, mascota):
        self._numero = numero
        self._nombre = nombre
        self._antiguedad = antiguedad
        self._mascota = mascota        # composición: el cliente TIENE UNA mascota

    @property
    def numero(self):
        return self._numero

    @property
    def nombre(self):
        return self._nombre

    @property
    def antiguedad(self):
        return self._antiguedad

    @antiguedad.setter
    def antiguedad(self, valor):
        self._antiguedad = valor       # en la solución de la cátedra este setter pisaba el
número

    @property
    def mascota(self):
        return self._mascota

    def __str__(self):
        return f"Cliente {self._numero}: {self._nombre} - {self._antiguedad} años de cliente -
Mascota: {self._mascota}"
```

principal.py

```
from cliente import Cliente
from mascota import Mascota


def main():
    clientes = []
    numero = int(input("Número de cliente (0 para terminar): "))
    while numero != 0:
        nombre = input("Nombre del cliente: ")
        antiguedad = int(input("Antigüedad (años): "))
        nombre_mascota = input("Nombre de la mascota: ")
        edad_mascota = int(input("Edad de la mascota: "))
        clientes.append(Cliente(numero, nombre, antiguedad, Mascota(nombre_mascota,
edad_mascota)))
        numero = int(input("Número de cliente (0 para terminar): "))

    if not clientes:
        print("No se cargó ningún cliente")
        return

    print("\nCantidad de clientes:", len(clientes))
    promedio = sum(c.mascota.edad for c in clientes) / len(clientes)
    print(f"Promedio de edad de las mascotas: {promedio:.2f}")
    antiguos = len([c for c in clientes if c.antiguedad >= 5])     # "mayor o igual a 5"
    print("Clientes con 5 años o más de antigüedad:", antiguos)
    print("Clientes cuya mascota tiene más de 5 años:")
    for c in clientes:
        if c.mascota.edad > 5:
            print("  ", c)


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Número de cliente (0 para terminar): 1
Nombre del cliente: Marta
Antigüedad (años): 6
Nombre de la mascota: Toby
Edad de la mascota: 8
Número de cliente (0 para terminar): 2
Nombre del cliente: Pablo
Antigüedad (años): 2
Nombre de la mascota: Luna
Edad de la mascota: 3
Número de cliente (0 para terminar): 3
Nombre del cliente: Carla
Antigüedad (años): 5
Nombre de la mascota: Simón
Edad de la mascota: 7
Número de cliente (0 para terminar): 0

Cantidad de clientes: 3
Promedio de edad de las mascotas: 6.00
Clientes con 5 años o más de antigüedad: 2
Clientes cuya mascota tiene más de 5 años:
   Cliente 1: Marta - 6 años de cliente - Mascota: Toby (8 años)
   Cliente 3: Carla - 5 años de cliente - Mascota: Simón (7 años)
```

04 Autos

Objetos | Solución propuesta (la cátedra no trae)

Enunciado (tal cual)

Diseñar una clase que permita modelar el funcionamiento básico de un automóvil, tal como se indica a continuación:

- conducir: esta opción debe recibir la cantidad de kilómetros a recorrer e informar si pueden ser recorridos y en tal caso descontar el combustible utilizado del tanque (se conoce que, por especificaciones técnicas del vehículo, con un litro recorre 11 km). Caso contrario informar que no hay combustible suficiente para el recorrido indicado.
- cargar combustible: está opción debe reponer combustible en el tanque tantos litros como se reciban como argumento pero teniendo en cuenta que el tanque tiene una capacidad de 49 lts más una reserva de 5 lts., en el caso de que los litros informados superen la capacidad del tanque, devolver la cantidad de lts de combustible derramados.

Qué practica

Guardar el estado dentro del objeto y que los métodos apliquen las reglas (consumo, capacidad y reserva).

Para tener en cuenta

- El tanque admite hasta 49 + 5 = 54 litros: lo que pase de ahí se derrama.

auto.py

```
class Auto:
    KM_POR_LITRO = 11
    CAPACIDAD = 49          # litros del tanque
    RESERVA = 5             # litros de reserva

    def __init__(self, combustible=0):
        self._combustible = min(combustible, self.CAPACIDAD + self.RESERVA)

    @property
    def combustible(self):
        return self._combustible

    def conducir(self, km):
        """Si alcanza el combustible, lo descuenta y devuelve True; si no, devuelve False."""
        litros = km / self.KM_POR_LITRO
        if litros > self._combustible:
            return False
        self._combustible -= litros
        return True

    def cargar_combustible(self, litros):
        """Carga hasta llenar tanque + reserva y devuelve los litros derramados (0 si no
sobró)."""
        lugar = self.CAPACIDAD + self.RESERVA - self._combustible
        cargados = min(litros, lugar)
        self._combustible += cargados
        return litros - cargados

    def __str__(self):
        return f"Auto con {self._combustible:.2f} litros"
```

principal.py

```
from auto import Auto


def main():
    auto = Auto()
    print(auto)

    derramados = auto.cargar_combustible(40)
    print(f"Carga 40 litros -> derramados: {derramados}. {auto}")

    for km in (300, 200):
        if auto.conducir(km):
            print(f"Recorre {km} km. {auto}")
        else:
            print(f"No hay combustible suficiente para {km} km. {auto}")

    derramados = auto.cargar_combustible(60)
    print(f"Carga 60 litros -> derramados: {derramados:.2f}. {auto}")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Auto con 0.00 litros
Carga 40 litros -> derramados: 0. Auto con 40.00 litros
Recorre 300 km. Auto con 12.73 litros
No hay combustible suficiente para 200 km. Auto con 12.73 litros
Carga 60 litros -> derramados: 18.73. Auto con 54.00 litros
```

05 Ascensor

Objetos | Solución de la cátedra

Enunciado (tal cual)

Un ascensor posee una capacidad máxima de n personas y está instalado en un edificio cuyos pisos se encuentran numerados. Se necesita desarrollar una clase Ascensor que represente el funcionamiento del mismo y que posea métodos para:

- Desplazarse a un piso determinado
- Subir personas
- Bajar personas
- Informar el piso donde se encuentra y la cantidad de personas que hay adentro

Los métodos deben verificar que el estado siempre sea correcto:

- Si se solicitar ir a un piso debe retornar verdadero si el piso existe y falso si no. Por ejemplo, si el ascensor puede viajar entre los pisos -2 y 10, debe retornar falso si le solicitan ir a un piso fuera de ese rango.
- Si la cantidad de personas que quieren subir supera la capacidad, debe retornar la cantidad de personas que efectivamente entren.
- Si la cantidad de personas que quieren bajar supera a la cantidad de personas que efectivamente hay adentro del ascensor, debe retornar esa última cantidad, es decir, que todos salen.
- Si para subir o bajar el método recibe una cantidad no válida, debe retornar -1.
- El método toString debe informar el piso en que se encuentra y la cantidad de personas que hay adentro en ese momento.

Qué practica

Métodos que validan el estado y lo informan con el valor que devuelven (`True` / `False`, cantidad o `-1`). Trae **tests de pytest**.

Para tener en cuenta

- El "método toString" del enunciado en Python es `__str__`.
- El ascensor arranca siempre en la planta baja (piso 0), así que el rango de pisos tiene que incluir el 0.

ascensor.py

```
from __future__ import annotations

class Ascensor:
    """
    Representa un ascensor con capacidad máxima y rango de pisos.

    Reglas (según README):
    - ir_a_piso(p) => True si p en [piso_min, piso_max] y se mueve; False si no existe.
    - subir(n):
        * retorna -1 si n <= 0
        * si n > espacio disponible, suben solo las que entran; retorna esa cantidad efectiva
```

```
    - bajar(n):
        * retorna -1 si n <= 0
        * si n > personas dentro, bajan todas; retorna cantidad efectiva
    - str(ascensor) informa piso actual y personas dentro.
    """

    def __init__(self, piso_min: int, piso_max: int, capacidad_max: int) -> None:
        if piso_min > piso_max:
            raise ValueError("piso_min no puede ser mayor que piso_max")
        if capacidad_max <= 0:
            raise ValueError("capacidad_max debe ser positiva")
        self._piso_min = int(piso_min)
        self._piso_max = int(piso_max)
        self._capacidad_max = int(capacidad_max)
        # El ascensor siempre arranca en 0 (planta baja)
        if 0 < self._piso_min or 0 > self._piso_max:
            raise ValueError("El piso inicial 0 está fuera del rango permitido")
        self._piso_actual = 0
        self._personas = 0

    @property
    def piso_min(self) -> int:
        return self._piso_min

    @property
    def piso_max(self) -> int:
        return self._piso_max

    @property
    def capacidad_max(self) -> int:
        return self._capacidad_max

    @property
    def piso_actual(self) -> int:
        return self._piso_actual

    @property
    def personas(self) -> int:
        return self._personas

    def ir_a_piso(self, piso_destino: int) -> bool:
        if piso_destino < self._piso_min or piso_destino > self._piso_max:
            return False
        self._piso_actual = int(piso_destino)
        return True

    def subir(self, cantidad: int) -> int:
        if cantidad is None or cantidad <= 0:
            return -1
        espacio_disponible = self._capacidad_max - self._personas
        suben = cantidad if cantidad <= espacio_disponible else espacio_disponible
        self._personas += suben
        return suben

    def bajar(self, cantidad: int) -> int:
        if cantidad is None or cantidad <= 0:
            return -1
        bajan = cantidad if cantidad <= self._personas else self._personas
        self._personas -= bajan
        return bajan
```

```
    def __str__(self) -> str:
        return f"Ascensor(piso={self._piso_actual}, personas={self._personas})"
```

demo.py

```
from ascensor import Ascensor

if __name__ == "__main__":
    a = Ascensor(-2, 10, 5)
    print(a)
    print("ir_a_piso(3)", a.ir_a_piso(3))
    print(a)
    print("ir_a_piso(15)", a.ir_a_piso(15))
    print(a)
    print("subir(3)", a.subir(3))
    print(a)
    print("subir(4)", a.subir(4))
    print(a)
    print("subir(0)", a.subir(0))
    print(a)
    print("bajar(2)", a.bajar(2))
    print(a)
    print("bajar(10)", a.bajar(10))
    print(a)
    print("bajar(-1)", a.bajar(-1))
    print(a)
```

test_ascensor.py

```
import pytest
from ascensor import Ascensor


@pytest.fixture()
def a():
    return Ascensor(-2, 10, 5)


def test_ir_a_piso_valido(a):
    assert a.ir_a_piso(3) is True
    assert a.piso_actual == 3


def test_ir_a_piso_invalido(a):
    # al fallar no cambia el piso actual (sigue en 0 si no se movió antes)
    assert a.ir_a_piso(15) is False
    assert a.piso_actual == 0


def test_subir_personas(a):
    assert a.subir(3) == 3
    assert a.personas == 3
    assert a.subir(4) == 2  # solo entran 2 (capacidad 5)
    assert a.personas == 5


def test_subir_invalido(a):
    assert a.subir(0) == -1
    assert a.subir(-3) == -1
    assert a.personas == 0
```

```python
def test_bajar_personas(a):
    a.subir(4)
    assert a.bajar(2) == 2
    assert a.personas == 2
    assert a.bajar(10) == 2  # bajan todas las que quedan
    assert a.personas == 0


def test_bajar_invalido(a):
    a.subir(2)
    assert a.bajar(0) == -1
    assert a.bajar(-1) == -1
    assert a.personas == 2



def test_constructor_piso_inicial_cero_dentro_de_rango():
    a1 = Ascensor(-2, 10, 5)
    assert a1.piso_actual == 0
    # 0 no está en rango [-10, -1], debe lanzar error
    with pytest.raises(ValueError):
        Ascensor(-10, -1, 3)
    # también debe lanzar error si el rango es [1, 10]
    with pytest.raises(ValueError):
        Ascensor(1, 10, 4)


def test_constructor_parametros_invalidos():
    with pytest.raises(ValueError):
        Ascensor(5, -1, 5)
    with pytest.raises(ValueError):
        Ascensor(-2, 10, 0)
```

✓ Pasa los **8** tests de la cátedra (`test_ascensor.py`)

Salida de demo.py

```
Ascensor(piso=0, personas=0)
ir_a_piso(3) True
Ascensor(piso=3, personas=0)
ir_a_piso(15) False
Ascensor(piso=3, personas=0)
subir(3) 3
Ascensor(piso=3, personas=3)
subir(4) 2
Ascensor(piso=3, personas=5)
subir(0) -1
Ascensor(piso=3, personas=5)
bajar(2) 2
Ascensor(piso=3, personas=3)
bajar(10) 3
Ascensor(piso=3, personas=0)
bajar(-1) -1
Ascensor(piso=3, personas=0)
```

Salida de python -m pytest -v

```
test_ascensor.py::test_ir_a_piso_valido PASSED
test_ascensor.py::test_ir_a_piso_invalido PASSED
test_ascensor.py::test_subir_personas PASSED
test_ascensor.py::test_subir_invalido PASSED
test_ascensor.py::test_bajar_personas PASSED
test_ascensor.py::test_bajar_invalido PASSED
test_ascensor.py::test_constructor_piso_inicial_cero_dentro_de_rango PASSED
test_ascensor.py::test_constructor_parametros_invalidos PASSED
============================== 8 passed in 0.03s ===============================
```

06 Partido de fútbol

Objetos | Solución propuesta (la cátedra no trae)

Enunciado (tal cual)

Un partido de fútbol se juega entre dos equipos denominados local y visitante. Desarrollar una clase Partido que contenga referencias a dos objetos de la clase Equipo. La clase Partido debe poseer atributos para registrar ademas la cantidad de goles realizada por cada equipo y métodos para informar cuál es el ganador o si hubo empate. Por otro lado, la clase Equipo debe poseer un método RegistrarPartido que reciba por parámetro un partido y asigne atributos (almacenados o calculados) para informar:

- Cantidad de partidos jugados
- Cantidad de partidos jugados como local
- Cantidad de partidos jugados como visitante
- Suma de goles realizados
- Suma de goles recibidos
- Diferencia total de goles
- Cantidad de partidos ganados
- Cantidad de partidos empatados
- Cantidad de partidos recibidos
- Sumatoria de puntos (suponiendo que todos los partidos son de un mismo campeonato)

Qué practica

Asociación entre objetos (el partido referencia a dos equipos) y atributos calculados con `@property`.

Para tener en cuenta

- Donde el enunciado dice “partidos recibidos” se interpreta **perdidos**.
- Puntos: 3 por partido ganado y 1 por empate.

partido.py

```
class Partido:
    def __init__(self, local, visitante, goles_local, goles_visitante):
        self.local = local              # referencias a dos objetos Equipo
        self.visitante = visitante
        self.goles_local = goles_local
        self.goles_visitante = goles_visitante

    def ganador(self):
        """Devuelve el equipo ganador, o None si fue empate."""
        if self.goles_local > self.goles_visitante:
            return self.local
        if self.goles_visitante > self.goles_local:
            return self.visitante
        return None

    def __str__(self):
        g = self.ganador()
        resultado = f"ganó {g.nombre}" if g else "empate"
        return f"{self.local.nombre} {self.goles_local} - {self.goles_visitante}
{self.visitante.nombre} ({resultado})"
```

equipo.py

```
class Equipo:
    def __init__(self, nombre):
        self.nombre = nombre
        self.jugados = 0
        self.de_local = 0
        self.de_visitante = 0
        self.goles_a_favor = 0
        self.goles_en_contra = 0
        self.ganados = 0
        self.empatados = 0
        self.perdidos = 0

    @property
    def diferencia_goles(self):          # calculado, no almacenado
        return self.goles_a_favor - self.goles_en_contra

    @property
    def puntos(self):                    # 3 por partido ganado, 1 por empate
        return self.ganados * 3 + self.empatados

    def registrar_partido(self, partido):
        if self is partido.local:
            self.de_local += 1
            a_favor, en_contra = partido.goles_local, partido.goles_visitante
        elif self is partido.visitante:
            self.de_visitante += 1
            a_favor, en_contra = partido.goles_visitante, partido.goles_local
        else:
            raise ValueError(f"{self.nombre} no jugó ese partido")
        self.jugados += 1
        self.goles_a_favor += a_favor
        self.goles_en_contra += en_contra
        ganador = partido.ganador()
        if ganador is None:
            self.empatados += 1
        elif ganador is self:
            self.ganados += 1
        else:
            self.perdidos += 1

    def __str__(self):
        return (f"{self.nombre}: PJ {self.jugados} (L {self.de_local} / V {self.de_visitante}) ·
"
                f"G {self.ganados} E {self.empatados} P {self.perdidos} · "
                f"GF {self.goles_a_favor} GC {self.goles_en_contra} Dif
{self.diferencia_goles:+d} · {self.puntos} pts")
```

principal.py

```
from equipo import Equipo
from partido import Partido


def main():
    belgrano, talleres, instituto = Equipo("Belgrano"), Equipo("Talleres"), Equipo("Instituto")
    partidos = [
        Partido(belgrano, talleres, 2, 1),
        Partido(talleres, instituto, 0, 0),
        Partido(instituto, belgrano, 3, 1),
        Partido(talleres, belgrano, 2, 2),
    ]
    for p in partidos:
        print(p)
        p.local.registrar_partido(p)
        p.visitante.registrar_partido(p)

    print("\nTabla:")
    for e in sorted([belgrano, talleres, instituto], key=lambda e: (e.puntos,
e.diferencia_goles), reverse=True):
        print("  ", e)


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Belgrano 2 - 1 Talleres (ganó Belgrano)
Talleres 0 - 0 Instituto (empate)
Instituto 3 - 1 Belgrano (ganó Instituto)
Talleres 2 - 2 Belgrano (empate)

Tabla:
   Instituto: PJ 2 (L 1 / V 1) · G 1 E 1 P 0 · GF 3 GC 1 Dif +2 · 4 pts
   Belgrano: PJ 3 (L 1 / V 2) · G 1 E 1 P 1 · GF 5 GC 6 Dif -1 · 4 pts
   Talleres: PJ 3 (L 2 / V 1) · G 0 E 2 P 1 · GF 3 GC 4 Dif -1 · 2 pts
```

07 Cuentas bancarias

Herencia | Solución de la cátedra, corregida

Enunciado (tal cual)

En un banco hay dos tipos de cuentas: cajas de ahorro y cuentas corrientes. Programar una clase Cuenta y dos derivadas correspondientes a cada uno de los tipos de cuenta. La clase Cuenta debe poseer dos métodos para operar con las mismas llamados Depositar y Extraer. Las cajas de ahorro no permiten extracciones si el saldo no es suficiente, mientras que las cuentas corrientes poseen un atributo “acuerdo” que indica el máximo saldo negativo admisible. La clase cuenta debe poseer al menos los atributos número, nombre y saldo.

Con las clases de las cuentas bancarias realizar un programa que permita ingresar un conjunto de cuentas bancarias en un arreglo. Luego de la carga el programa debe permitir ingresar operaciones de depósitos y extracciones en las cuentas. Al finalizar la carga de las operaciones el programa debe mostrar el listado de las cuentas con sus saldos definitivos.

Qué practica

Herencia y **clase abstracta**: cada tipo de cuenta implementa `extraer` a su manera (polimorfismo).

Qué se corrigió de la solución de la cátedra

- La solución de la cátedra era solo una prueba con dos cuentas fijas: acá está el programa que pide el enunciado (carga de cuentas, operaciones y listado final).
- `extraer` devuelve si se pudo hacer, y el programa avisa; antes imprimía desde adentro de la clase.

cuentas.py

```python
from abc import ABC, abstractmethod


class Cuenta(ABC):
    def __init__(self, numero, nombre, saldo=0):
        self.numero = numero
        self.nombre = nombre
        self.saldo = saldo

    def depositar(self, monto):
        self.saldo += monto

    @abstractmethod
    def extraer(self, monto):
        """Cada tipo de cuenta decide si puede extraer. Devuelve True si se hizo."""

    def __str__(self):
        return f"{type(self).__name__} {self.numero} - {self.nombre} - saldo ${self.saldo:,.2f}"


class CajaAhorro(Cuenta):
    def extraer(self, monto):
        if monto > self.saldo:           # no permite quedar en negativo
            return False
        self.saldo -= monto
        return True


class CuentaCorriente(Cuenta):
    def __init__(self, numero, nombre, saldo=0, acuerdo=0):
        super().__init__(numero, nombre, saldo)
        self.acuerdo = acuerdo            # máximo saldo negativo admitido

    def extraer(self, monto):
        if self.saldo - monto < -self.acuerdo:
            return False
        self.saldo -= monto
        return True
```

principal.py

```python
from cuentas import CajaAhorro, CuentaCorriente


def cargar_cuentas():
    cuentas = []
    tipo = int(input("Tipo de cuenta (1: caja de ahorro, 2: cuenta corriente, 0: terminar): "))
    while tipo != 0:
        numero = int(input("Número: "))
        nombre = input("Titular: ")
        saldo = float(input("Saldo inicial: "))
        if tipo == 1:
            cuentas.append(CajaAhorro(numero, nombre, saldo))
        else:
            acuerdo = float(input("Acuerdo (saldo negativo máximo): "))
            cuentas.append(CuentaCorriente(numero, nombre, saldo, acuerdo))
        tipo = int(input("Tipo de cuenta (1: caja de ahorro, 2: cuenta corriente, 0: terminar):
"))
    return cuentas
```

```python
def buscar(cuentas, numero):
    for c in cuentas:
        if c.numero == numero:
            return c
    return None


def operar(cuentas):
    numero = int(input("Cuenta a operar (0 para terminar): "))
    while numero != 0:
        cuenta = buscar(cuentas, numero)
        if cuenta is None:
            print("No existe esa cuenta")
        else:
            operacion = input("D: depositar / E: extraer: ").upper()
            monto = float(input("Monto: "))
            if operacion == "D":
                cuenta.depositar(monto)
            elif not cuenta.extraer(monto):      # polimorfismo: cada cuenta extrae a su manera
                print("No se puede extraer: saldo insuficiente")
        numero = int(input("Cuenta a operar (0 para terminar): "))


def main():
    cuentas = cargar_cuentas()
    operar(cuentas)
    print("\nSaldos finales:")
    for c in cuentas:
        print("  ", c)


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Tipo de cuenta (1: caja de ahorro, 2: cuenta corriente, 0: terminar): 1
Número: 100
Titular: Ana
Saldo inicial: 5000
Tipo de cuenta (1: caja de ahorro, 2: cuenta corriente, 0: terminar): 2
Número: 200
Titular: Juan
Saldo inicial: 1000
Acuerdo (saldo negativo máximo): 20000
Tipo de cuenta (1: caja de ahorro, 2: cuenta corriente, 0: terminar): 0
Cuenta a operar (0 para terminar): 100
D: depositar / E: extraer: E
Monto: 8000
No se puede extraer: saldo insuficiente
Cuenta a operar (0 para terminar): 200
D: depositar / E: extraer: E
Monto: 15000
Cuenta a operar (0 para terminar): 100
D: depositar / E: extraer: D
Monto: 2000
Cuenta a operar (0 para terminar): 200
D: depositar / E: extraer: E
Monto: 10000
No se puede extraer: saldo insuficiente
Cuenta a operar (0 para terminar): 999
No existe esa cuenta
Cuenta a operar (0 para terminar): 100
D: depositar / E: extraer: E
Monto: 7000
Cuenta a operar (0 para terminar): 0

Saldos finales:
   CajaAhorro 100 - Ana - saldo $0.00
   CuentaCorriente 200 - Juan - saldo $-14,000.00
```

08 Plantel de empleados

Polimorfismo | Solución de la cátedra, corregida

Enunciado (tal cual)

Plantel de empleados

Leer todo el archivo de empleados. El mismo posee los siguientes datos:

- Tipo de empleado (1: obrero, 2: administrativo, 3: vendedor).
- Legajo
- Nombre
- Apellido
- Sueldo básico
- La tercera columna depende del tipo de empleado, en el caso de los obreros indica la cantidad de días trabajados en el mes, en el caso de los administrativos indica si le corresponde cobrar presentismo y en el caso de los vendedores indica el importe total de ventas realizadas en mes.

Por cada tipo de empleados el sueldo se calcula de una forma diferente:

- En el caso de los obreros, el sueldo básico corresponde a un mes de 20 días laborables, y el sueldo a cobrar debe ser un propocional a la cantidad de días efectivamente trabajados.
- En el caso de los administrativos, el sueldo a cobrar es igual al básico, pero se incrementa en 13% si le corresponde cobrar el presentismo.
- Los vendedores cobrar el sueldo básico más un 1% del total vendido.

Luego de la carga:

- Calcular el total a pagar de sueldos
- Contar los empleados por tipo (tres totales)
- Buscar un empleado por legajo y mostrar el sueldo a pagar

Qué practica

Polimorfismo con datos de un archivo: cada empleado calcula su sueldo (`neto`) y la empresa los suma sin preguntar el tipo.

Qué se corrigió de la solución de la cátedra

- El obrero dividía el básico por **22** días, pero el enunciado dice que corresponde a **20**: los sueldos de los obreros daban de menos.
- `Empleado` ahora es abstracta (`neto` y `tipo` obligatorios en cada subclase) y el archivo se abre con `with`.

empleado.py

```python
from abc import ABC, abstractmethod


class Empleado(ABC):
    def __init__(self, legajo, nombre, apellido, basico):
        self.legajo = legajo
        self.nombre = nombre
        self.apellido = apellido
        self.basico = basico

    @property
    @abstractmethod
    def neto(self):
        """Sueldo a cobrar: cada tipo de empleado lo calcula distinto."""

    @property
    @abstractmethod
    def tipo(self):
        """1: obrero, 2: administrativo, 3: vendedor"""

    def __str__(self):
        return f"{self.legajo} {self.nombre} {self.apellido} ({type(self).__name__}) - cobra
${self.neto:,.2f}"
```

obrero.py

```python
from empleado import Empleado


class Obrero(Empleado):
    DIAS_DEL_MES = 20        # el básico corresponde a 20 días (la solución de la cátedra usaba
22)

    def __init__(self, legajo, nombre, apellido, basico, dias):
        super().__init__(legajo, nombre, apellido, basico)
        self.dias = dias

    @property
    def neto(self):
        return self.basico / self.DIAS_DEL_MES * self.dias

    @property
    def tipo(self):
        return 1
```

administrativo.py

```python
from empleado import Empleado


class Administrativo(Empleado):
    def __init__(self, legajo, nombre, apellido, basico, presentismo):
        super().__init__(legajo, nombre, apellido, basico)
        self.presentismo = presentismo

    @property
    def neto(self):
        return self.basico * 1.13 if self.presentismo else self.basico

    @property
    def tipo(self):
        return 2
```

vendedor.py

```python
from empleado import Empleado


class Vendedor(Empleado):
    def __init__(self, legajo, nombre, apellido, basico, ventas):
        super().__init__(legajo, nombre, apellido, basico)
        self.ventas = ventas

    @property
    def neto(self):
        return self.basico + self.ventas * 0.01

    @property
    def tipo(self):
        return 3
```

empresa.py

```python
from obrero import Obrero
from administrativo import Administrativo
from vendedor import Vendedor


def empleado_desde_linea(linea):
    """tipo;legajo;nombre;apellido;básico;dato del tipo"""
    tipo, legajo, nombre, apellido, basico, extra = linea.strip().split(";")
    legajo, basico = int(legajo), float(basico)
    if tipo == "1":
        return Obrero(legajo, nombre, apellido, basico, int(extra))
    if tipo == "2":
        return Administrativo(legajo, nombre, apellido, basico, extra == "true")
    return Vendedor(legajo, nombre, apellido, basico, float(extra))


class Empresa:
    def __init__(self):
        self._plantel = {}                      # legajo -> empleado

    def leer_archivo(self, nombre_archivo):
        with open(nombre_archivo, encoding="utf-8") as archivo:
            for linea in archivo:
                if linea.strip():
                    e = empleado_desde_linea(linea)
                    self._plantel[e.legajo] = e

    def cantidad_empleados(self):
        return len(self._plantel)

    def total_sueldos(self):
        return sum(e.neto for e in self._plantel.values())      # polimorfismo: cada uno su neto

    def cantidad_por_tipo(self):
        cant = [0, 0, 0]
        for e in self._plantel.values():
            cant[e.tipo - 1] += 1
        return cant

    def buscar(self, legajo):
        return self._plantel.get(legajo)
```

principal.py

```python
from empresa import Empresa


def main():
    empresa = Empresa()
    empresa.leer_archivo("empleados.csv")
    print("Empleados leídos:", empresa.cantidad_empleados())
    print(f"Total a pagar de sueldos: ${empresa.total_sueldos():,.2f}")
    obreros, administrativos, vendedores = empresa.cantidad_por_tipo()
    print(f"Obreros: {obreros} · Administrativos: {administrativos} · Vendedores: {vendedores}")

    legajo = int(input("Legajo a buscar: "))
    empleado = empresa.buscar(legajo)
    print(empleado if empleado else "No existe ningún empleado con ese legajo")


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Empleados leídos: 2000
Total a pagar de sueldos: $125,958,245.00
Obreros: 505 · Administrativos: 455 · Vendedores: 1040
Legajo a buscar: 2870
2870 ESTEBAN MARQUEZ (Vendedor) - cobra $23,661.00
```

09 Hospital

Polimorfismo | Solución de la cátedra, corregida

Enunciado (tal cual)

Un hospital de nuestra ciudad necesita conocer cierta información respecto a los importes cobrados por las distintas atenciones que realiza,y requiere de un sistema orientado a objetos que le dé soporte a ello. Los datos relevantes son los siguientes:

- Del Hospital sólo interesa registrar la razón social; y una colección con todas las atenciones realizadas.
- Cada atención tiene los siguientes datos: un código numérico que identifica a cada atención; y un valor numérico que representa el tipode cobro (1: “efectivo”; 2: “tarjeta de crédito”). Las atenciones pueden ser médicas o de farmacia:
  - Una atención médica agrega los siguientes datos: el paciente atendido; y el importe de la consulta.
  - Una atención de farmacia agregan los siguientes datos: el importe total de los medicamentos vendidos en dicha atención; y un cupón de descuento que especifica el monto de descuento que se aplicaría sobre el importe total de los medicamentos; en caso que el cupón de descuento sea igual que 0, no se realizará ningún descuento; cabe aclarar que el cupón de descuento debe ser 0 o positivo.
  - Del paciente asociado a la atención médica se registran: su nombre; el síntoma que prevalece (1: “corazon”, 2: “pulmon”, 3: “otras”); y un valor booleano que representa si el paciente es habitual (true) o no (false) del hospital.

De cada clase de atención se debe calcular un importeACobrar en pesos, que representa lo que el hospital cobra al paciente por la atención brindada. Este cálculo dependerá del tipo de cobro de la atención y de la clase de atención brindada:

- El importe total de las atenciones médicas se calcula a partir del importe de la consulta. En caso que el paciente sea “habitual” se leaplicará un descuento del 25%. Por último, si el tipo de cobro es igual a 2 (“tarjeta de crédito”) se le incrementa al anterior importe un20% más; en caso que el cobro sea igual a 1 (“efectivo”) se le realiza un descuento del 10% sobre el anterior importe.
- El importe total de las atenciones de farmacia se calcula a partir del importe total de los medicamentos vendidos. Luego se le realiza el descuento del cupón de descuento cuyo importe está especificado. Por último, si el tipo de cobro es igual a 2 (“tarjeta de crédito”) se leincrementa al anterior importe un 30% más; en caso que el cobro sea igual a 1 (“efectivo”) se le realiza un descuento del 5% sobre elanterior importe.

Con lo expuesto anteriormente, usted deberá implementar:

- Todas las clases del modelo presentado.
- Los siguientes requerimientos de métodos:
  - Para las clases Atención, Médica, Farmacia y Paciente implementar su constructor, acceso y modificación y toString.
  - Para la clase Hospital implementar el método addAtención que agrega una atención a la colección.
- Definir e implementar los métodos importeACobrar de las atenciones, que calcule y devuelva el importe a cobrar por la atención de acuerdo a los criterios anteriormente especificados.
- En clase Hospital se requiere la implementación de los siguientes métodos:

- importe_total_atencion_consulta: debe calcular la suma de los importes de las consultas de las atenciones médicas.
- importe_promedio_atenciones: debe calcular el promedio de los importes a cobrar por aquellas atenciones médicas cuyo importe a cobrar se encuentre entre dos valores recibidos como parámetros.
- codigo_primera_atencion_habitual: debe retornar el código de la primera atención médica que se haya registrado para un paciente habitual, o 0 si no existe ninguna.

Qué practica

Una colección con objetos de distintas clases (atenciones médicas y de farmacia) que responden al mismo mensaje.

Qué se corrigió de la solución de la cátedra

- `Farmacia.importe_a_cobrar` no tenía `return`: devolvía `None`.
- Faltaba validar que el cupón sea 0 o positivo, y el síntoma es un código (1, 2 o 3) como pide el enunciado.
- La prueba tenía una sola atención; ahora hay cinco, de los dos tipos, para ver todos los cálculos.

Para tener en cuenta

- Importes de la salida, a mano: médica 1 = 10.000 × 0,9 = 9.000 · médica 2 = 10.000 × 0,75 × 1,2 = 9.000 · farmacia 3 = (8.000 − 1.000) × 0,95 = 6.650 · farmacia 4 = 5.000 × 1,3 = 6.500 · médica 5 = 20.000 × 0,75 × 0,9 = 13.500.

También en la Guía del 1º parcial

El simulacro **Hospital** es este mismo problema con datos en CSV y **tests de pytest**, con la resolución de la cátedra.

atencion.py

```
from abc import ABC, abstractmethod

EFECTIVO, TARJETA = 1, 2


class Atencion(ABC):
    def __init__(self, codigo, tipo_cobro):
        self.codigo = codigo
        self.tipo_cobro = tipo_cobro            # 1: efectivo, 2: tarjeta de crédito

    @abstractmethod
    def importe_a_cobrar(self):
        """Cada tipo de atención calcula su importe."""

    def __str__(self):
        cobro = "efectivo" if self.tipo_cobro == EFECTIVO else "tarjeta"
        return f"{type(self).__name__} {self.codigo} ({cobro}) ->
${self.importe_a_cobrar():,.2f}"
```

paciente.py

```python
SINTOMAS = {1: "corazón", 2: "pulmón", 3: "otras"}


class Paciente:
    def __init__(self, nombre, sintoma, habitual):
        self.nombre = nombre
        self.sintoma = sintoma                  # 1, 2 o 3
        self.habitual = habitual                # True / False

    def __str__(self):
        return f"{self.nombre} ({SINTOMAS[self.sintoma]}{', habitual' if self.habitual else ''})"
```

medica.py

```python
from atencion import Atencion, TARJETA


class Medica(Atencion):
    def __init__(self, codigo, tipo_cobro, paciente, importe):
        super().__init__(codigo, tipo_cobro)
        self.paciente = paciente
        self.importe = importe                  # importe de la consulta

    def importe_a_cobrar(self):
        importe = self.importe
        if self.paciente.habitual:
            importe *= 0.75                     # 25 % de descuento
        return importe * (1.20 if self.tipo_cobro == TARJETA else 0.90)
```

farmacia.py

```python
from atencion import Atencion, TARJETA


class Farmacia(Atencion):
    def __init__(self, codigo, tipo_cobro, importe_medicamentos, cupon_descuento=0):
        super().__init__(codigo, tipo_cobro)
        if cupon_descuento < 0:
            raise ValueError("El cupón de descuento debe ser 0 o positivo")
        self.importe_medicamentos = importe_medicamentos
        self.cupon_descuento = cupon_descuento

    def importe_a_cobrar(self):
        importe = self.importe_medicamentos - self.cupon_descuento
        return importe * (1.30 if self.tipo_cobro == TARJETA else 0.95)   # faltaba el return
```

hospital.py

```
from medica import Medica


class Hospital:
    def __init__(self, razon_social):
        self.razon_social = razon_social
        self.atenciones = []

    def add_atencion(self, atencion):
        self.atenciones.append(atencion)

    def _medicas(self):
        return [a for a in self.atenciones if isinstance(a, Medica)]

    def importe_total_atencion_consulta(self):
        return sum(a.importe for a in self._medicas())

    def importe_promedio_atenciones(self, desde, hasta):
        importes = [a.importe_a_cobrar() for a in self._medicas() if desde < a.importe_a_cobrar()
< hasta]
        return sum(importes) / len(importes) if importes else 0

    def codigo_primera_atencion_habitual(self):
        for a in self._medicas():
            if a.paciente.habitual:
                return a.codigo
        return 0
```

principal.py

```
from hospital import Hospital
from medica import Medica
from farmacia import Farmacia
from paciente import Paciente
from atencion import EFECTIVO, TARJETA


def main():
    h = Hospital("Hospital Privado")
    h.add_atencion(Medica(1, EFECTIVO, Paciente("Juan", 3, False), 10000))
    h.add_atencion(Medica(2, TARJETA, Paciente("Ana", 1, True), 10000))
    h.add_atencion(Farmacia(3, EFECTIVO, 8000, 1000))
    h.add_atencion(Farmacia(4, TARJETA, 5000))
    h.add_atencion(Medica(5, EFECTIVO, Paciente("Luis", 2, True), 20000))

    for a in h.atenciones:
        print(a)
    print(f"\nTotal de consultas médicas: ${h.importe_total_atencion_consulta():,.2f}")
    print(f"Promedio de las médicas que cobran entre $5.000 y $10.000:
${h.importe_promedio_atenciones(5000, 10000):,.2f}")
    print("Primera atención médica de un paciente habitual:",
h.codigo_primera_atencion_habitual())


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Medica 1 (efectivo) -> $9,000.00
Medica 2 (tarjeta) -> $9,000.00
Farmacia 3 (efectivo) -> $6,650.00
Farmacia 4 (tarjeta) -> $6,500.00
Medica 5 (efectivo) -> $13,500.00

Total de consultas médicas: $40,000.00
Promedio de las médicas que cobran entre $5.000 y $10.000: $9,000.00
Primera atención médica de un paciente habitual: 2
```

10 Transporte de cargas (integrador)

Integrador | Solución de la cátedra, corregida

Enunciado (tal cual)

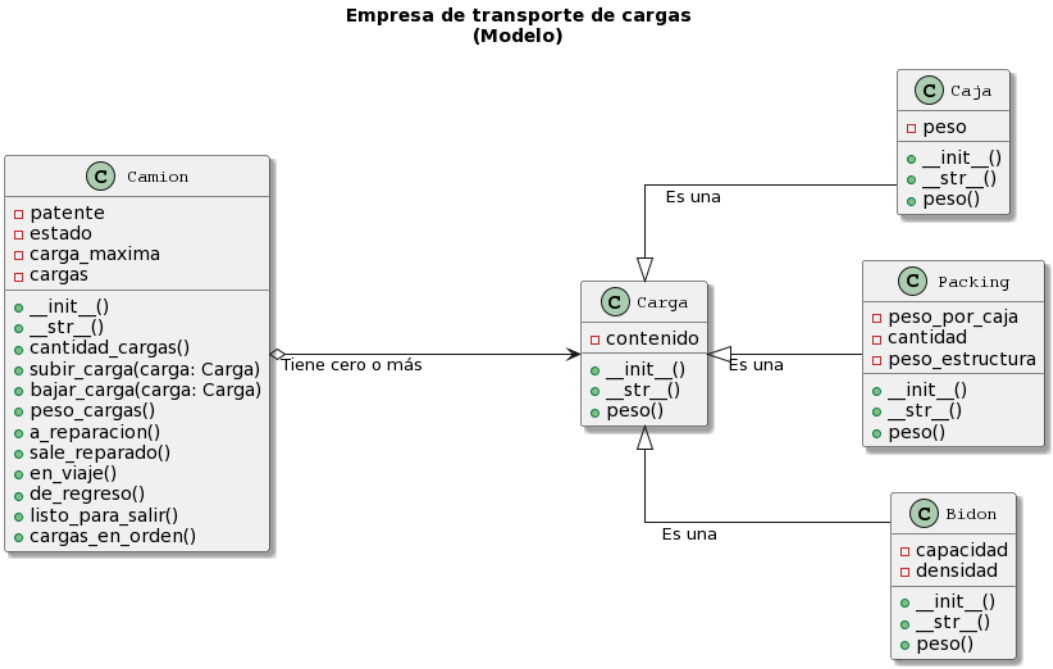
Empresa de transporte de cargas

Una empresa de transporte de cargas necesita un software que la ayude a organizarse con la carga de las camiones que maneja. La empresa puede recibir packings, cajas sueltas y bidones que transportan líquido.

- Un packing es una estructura de madera que arriba tiene un montón de cajas, se envuelve todo con plástico para que no se desbanden las cajas. Todas las cajas tiene el mismo peso.
  - El peso de un packing es (peso de cada caja * cantidad de cajas) + peso de la estructura de madera que va abajo.
  - Para cada packing se informa qué llevan las cajas, p.ej. Material de construcción.
- De cada caja suelta se informa el peso individualmente, son todas distintas.
- El peso de un bidón es su capacidad en litros por la densidad (o sea, cuántos kg pesa un litro) del líquido que se le carga. Los bidones van siempre llenos hasta el tope.

Cada camión puede llevar hasta una carga máxima medida en kg. Además, cada camión puede: estar disponible para la carga (en cuyo caso ya puede tener cosas cargadas), estar en reparación, o estar de viaje. Algunos requerimientos:

- Subir una carga al camión, donde la carga puede ser un packing, una caja suelta, o un bidón.
  - Considerar que no se puede saturar un camión con más peso de lo que su carga máxima permite.
- Bajar una carga del camión,
  - siempre que el camión se encuentre disponible con cargas y que a su vez la carga se encuentre presente dentro de él.
- Conocer el total de cargas de un camión en todo momento y su peso.
- Permitir modificar el estado de un camión, sea porque
  - entro en reparación,
  - salió de reparación,
  - está en viaje o
  - de regreso.
- Obtener el listado de cargas contenidas, como string, para imprimir por pantalla.
- Saber si un camión está listo para partir, que es:
  - Si está disponible para la carga,
  - y el peso total de lo que tiene cargado es de al menos 75% de su carga máxima.



El modelo de clases del enunciado

Qué practica

Todo junto: herencia, clase abstracta, composición, estados del camión, excepciones y lectura de CSV.

Qué se corrigió de la solución de la cátedra

- `en_reparacion` llamaba a `cantidad_cargas()` como función, pero es una propiedad: daba error.
- Los estados no coincidían (`"EN_REPARACION"` en un lado y `"EN REPARACION"` en otro) y faltaban salir de reparación, volver de viaje y "listo para partir" (75 % de la carga máxima).
- Se podía bajar carga con el camión en viaje, y no se podía cargar justo hasta el máximo (usaba `<` en vez de `<=`).
- En los bidones, el CSV dice capacidad y densidad, pero se pasaban al revés.
- `Carga.peso` no tenía `self`, y la carga se hacía al azar (`random`): ahora se cargan en orden y la salida es siempre la misma.

También en la Guía del 1º parcial

El simulacro **Empresa de transporte** es este mismo problema con datos en CSV y **tests de pytest**, con la resolución de la cátedra.

cargas.py

```python
from abc import ABC, abstractmethod


class Carga(ABC):
    def __init__(self, contenido):
        self.contenido = contenido

    @abstractmethod
    def peso(self):
        """Peso en kg: cada tipo de carga lo calcula distinto."""

    def __str__(self):
        return f"{type(self).__name__}: {self.contenido} ({self.peso():.2f} kg)"


class Caja(Carga):
    def __init__(self, contenido, peso):
        super().__init__(contenido)
        self._peso = peso

    def peso(self):
        return self._peso


class Packing(Carga):
    def __init__(self, contenido, peso_caja, cantidad, peso_estructura):
        super().__init__(contenido)
        self.peso_caja = peso_caja
        self.cantidad = cantidad
        self.peso_estructura = peso_estructura

    def peso(self):
        return self.peso_caja * self.cantidad + self.peso_estructura


class Bidon(Carga):
    def __init__(self, contenido, capacidad, densidad):
        super().__init__(contenido)
        self.capacidad = capacidad          # litros (siempre lleno)
        self.densidad = densidad            # kg por litro

    def peso(self):
        return self.capacidad * self.densidad
```

camion.py

```python
DISPONIBLE, EN_REPARACION, EN_VIAJE = "DISPONIBLE", "EN REPARACIÓN", "EN VIAJE"


class Camion:
    def __init__(self, patente, carga_maxima):
        self.patente = patente
        self.carga_maxima = carga_maxima      # kg
        self.estado = DISPONIBLE
        self._cargas = []                      # composición: el camión tiene cero o más cargas

    @property
    def cantidad_cargas(self):
        return len(self._cargas)
```

```
    @property
    def peso_cargas(self):
        return sum(c.peso() for c in self._cargas)    # polimorfismo: cada carga sabe su peso

    def subir_carga(self, carga):
        if self.estado != DISPONIBLE:
            raise ValueError(f"No se puede cargar: el camión está {self.estado}")
        if self.peso_cargas + carga.peso() > self.carga_maxima:
            raise ValueError(f"No entra {carga}: quedan {self.carga_maxima -
self.peso_cargas:.2f} kg libres")
        self._cargas.append(carga)

    def bajar_carga(self, carga):
        if self.estado != DISPONIBLE:
            raise ValueError(f"No se puede descargar: el camión está {self.estado}")
        if carga not in self._cargas:
            raise ValueError("Esa carga no está en el camión")
        self._cargas.remove(carga)

    def listo_para_partir(self):
        return self.estado == DISPONIBLE and self.peso_cargas >= 0.75 * self.carga_maxima

    # Cambios de estado
    def entrar_a_reparacion(self):
        self._cambiar(DISPONIBLE, EN_REPARACION)

    def salir_de_reparacion(self):
        self._cambiar(EN_REPARACION, DISPONIBLE)

    def salir_de_viaje(self):
        self._cambiar(DISPONIBLE, EN_VIAJE)

    def volver_de_viaje(self):
        self._cambiar(EN_VIAJE, DISPONIBLE)

    def _cambiar(self, desde, hasta):
        if self.estado != desde:
            raise ValueError(f"El camión está {self.estado}: no puede pasar a {hasta}")
        self.estado = hasta

    def listado_cargas(self):
        lineas = [f"Camión {self.patente} · {self.estado} · {self.cantidad_cargas} cargas · "
                  f"{self.peso_cargas:.2f} de {self.carga_maxima} kg"]
        lineas += [f"   {c}" for c in self._cargas]
        return "\n".join(lineas)
```

principal.py

```
import csv
from cargas import Caja, Packing, Bidon
from camion import Camion


def leer_cargas():
    """Lee las tres planillas y devuelve todas las cargas (en el orden de los archivos)."""
    cargas = []
    with open("Cajas.csv", encoding="utf-8") as f:
        for fila in csv.DictReader(f):
            cargas.append(Caja(fila["producto"], float(fila["peso"])))
    with open("Packing.csv", encoding="utf-8") as f:
        for fila in csv.DictReader(f):
            cargas.append(Packing(fila["producto"], float(fila["peso"]), int(fila["cantidad"]),
float(fila["estructura"])))
    with open("Bidones.csv", encoding="utf-8") as f:
        for fila in csv.DictReader(f):
            cargas.append(Bidon(fila["producto"], float(fila["capacidad"]),
float(fila["densidad"])))
    return cargas


def main():
    camion = Camion("KMS851", 200)
    subidas, rechazadas = [], []
    for carga in leer_cargas():
        try:
            camion.subir_carga(carga)
            subidas.append(carga)
        except ValueError as e:          # el camión no deja pasarse del peso máximo
            rechazadas.append(e)
    print(f"Subieron {len(subidas)} cargas y no entraron {len(rechazadas)}. Por ejemplo:")
    print("  ", rechazadas[0])

    print(camion.listado_cargas())
    print("¿Listo para partir?", "Sí" if camion.listo_para_partir() else "No")

    camion.salir_de_viaje()
    try:
        camion.bajar_carga(subidas[0])
    except ValueError as e:
        print("Intento de bajar una carga en viaje ->", e)
    camion.volver_de_viaje()
    print("Volvió:", camion.estado)


if __name__ == "__main__":
    main()
```

Salida al correrlo

```
Subieron 23 cargas y no entraron 27. Por ejemplo:
   No entra Packing: Pañuelos de papel en caja (39.00 kg): quedan 4.00 kg libres
Camión KMS851 · DISPONIBLE · 23 cargas · 199.00 de 200 kg
   Caja: Yogur (4.00 kg)
   Caja: Pollo (4.00 kg)
   Caja: Fresas (7.00 kg)
   Caja: Queso (14.00 kg)
   Caja: Pescado (7.00 kg)
   Caja: Queso (8.00 kg)
   Caja: Queso (12.00 kg)
   Caja: Pan (3.00 kg)
   Caja: Yogur (20.00 kg)
   Caja: Pan (3.00 kg)
   Caja: Arroz (7.00 kg)
   Caja: Pescado (4.00 kg)
   Caja: Arroz (2.00 kg)
   Caja: Pollo (6.00 kg)
   Caja: Lechuga (3.00 kg)
   Caja: Manzanas (3.00 kg)
   Caja: Pollo (6.00 kg)
   Caja: Fresas (12.00 kg)
   Caja: Arroz (18.00 kg)
   Caja: Pescado (12.00 kg)
   Packing: Bolsas de papas fritas en pack (18.00 kg)
   Packing: Latas de conservas en pack (23.00 kg)
   Packing: Cerveza en pack (3.00 kg)
¿Listo para partir? Sí
Intento de bajar una carga en viaje -> No se puede descargar: el camión está EN VIAJE
Volvió: DISPONIBLE
```

11 Ejemplos: manejo de excepciones

Excepciones · Solución de la cátedra

Qué muestra (no tiene enunciado)

Los ejemplos de la cátedra de manejo de errores: capturar uno en particular (`ZeroDivisionError`, `FileNotFoundError`), capturar cualquiera con `Exception`, el bloque `finally` que se ejecuta siempre y cómo mostrar dónde ocurrió con `traceback`.

Bloque	Cuándo se ejecuta
`try`	El código que puede fallar
`except Tipo as e`	Solo si ocurre ese error (o uno de sus hijos); `e` trae el mensaje
`else`	Solo si **no** hubo error
`finally`	**Siempre**, haya o no error (cerrar archivos, conexiones)
`raise`	Para lanzar un error propio: `raise ValueError("mensaje")`

excepciones.py

```
"""
Pruebas de excepciones
"""
import traceback

# División por cero

#a = 10 / 0

# Ahora con excepciones

def rutina_con_excepciones():
    try:
        a = 10
        b = 0
        c = a / b
    except ZeroDivisionError as e:
        print ("Ha ocurrido un problema:",e)
    else:
        print(a,"dividido",b,"es",c)
    finally:
        print("Esto se ejecuta haya o no excepciones")


    try:
        una_instruccion_absurda()
    except Exception as e:
        print ("Captura CUALQUIER excepción, por ejemplo una función inexistente", e)

    try:
        m = open("no_ta_file.txt")
        texto = m.read()
        print (texto)
    except FileNotFoundError as e:
        print ("El archivo no existe:", e)
        '''
        Si importamos el módulo traceback, podemos tener acceso
        a información del estado de ejecución al momento de dispararse
        una excepción, y realizar acciones con esa información,
        por ejemplo imprimir la pila de llamada.'''
        print (traceback.print_stack())

if __name__ == '__main__':
    rutina_con_excepciones()
```

Salida al correrlo

```
Ha ocurrido un problema: division by zero
Esto se ejecuta haya o no excepciones
Captura CUALQUIER excepción, por ejemplo una función inexistente name 'una_instruccion_absurda'
is not defined
El archivo no existe: [Errno 2] No such file or directory: 'no_ta_file.txt'
  File "excepciones.py", line 44, in <module>
    rutina_con_excepciones()
  File "excepciones.py", line 41, in rutina_con_excepciones
    print (traceback.print_stack())
None
```

Para tener en cuenta

- La última línea muestra `None` porque el ejemplo hace `print(traceback.print_stack())`: `print_stack()` ya imprime la pila y devuelve `None`, que el `print` de afuera muestra. Alcanza con `traceback.print_stack()`.

12 Testing con pytest

Testing | Solución de la cátedra

En el parcial los tests vienen dados y son el contrato. Una prueba de pytest es una función que empieza con `test_`, en un archivo que empieza con `test_`, y usa `assert`: si la condición es falsa, la prueba falla.

1. Instalar una vez: `pip install pytest`.
2. Parado en la carpeta del ejercicio: `python -m pytest` (con `-v` muestra cada prueba).
3. Si algo falla, pytest muestra la línea del `assert` y los dos valores comparados (lo que dio y lo que esperaba).

Ascensor

En `testing/ascensor` la cátedra deja la clase `Ascensor` vacía (`pass`) y los tests: hay que escribir la clase hasta que pasen. Los tests son los mismos del ejercicio 05, así que la solución es esa.

test_ascensor.py

```python
import pytest
from ascensor import Ascensor


@pytest.fixture()
def a():
    return Ascensor(-2, 10, 5)


def test_ir_a_piso_valido(a):
    assert a.ir_a_piso(3) is True
    assert a.piso_actual == 3


def test_ir_a_piso_invalido(a):
    # al fallar no cambia el piso actual (sigue en 0 si no se movió antes)
    assert a.ir_a_piso(15) is False
    assert a.piso_actual == 0


def test_subir_personas(a):
    assert a.subir(3) == 3
    assert a.personas == 3
    assert a.subir(4) == 2  # solo entran 2 (capacidad 5)
    assert a.personas == 5


def test_subir_invalido(a):
    assert a.subir(0) == -1
    assert a.subir(-3) == -1
    assert a.personas == 0


def test_bajar_personas(a):
    a.subir(4)
    assert a.bajar(2) == 2
    assert a.personas == 2
    assert a.bajar(10) == 2  # bajan todas las que quedan
    assert a.personas == 0
```

```python
def test_bajar_invalido(a):
    a.subir(2)
    assert a.bajar(0) == -1
    assert a.bajar(-1) == -1
    assert a.personas == 2



def test_constructor_piso_inicial_cero_dentro_de_rango():
    a1 = Ascensor(-2, 10, 5)
    assert a1.piso_actual == 0
    # 0 no está en rango [-10, -1], debe lanzar error
    with pytest.raises(ValueError):
        Ascensor(-10, -1, 3)
    # también debe lanzar error si el rango es [1, 10]
    with pytest.raises(ValueError):
        Ascensor(1, 10, 4)


def test_constructor_parametros_invalidos():
    with pytest.raises(ValueError):
        Ascensor(5, -1, 5)
    with pytest.raises(ValueError):
        Ascensor(-2, 10, 0)
```

✓ Pasa los **8** tests de la cátedra (`test_ascensor.py`) con el ascensor del ejercicio 05

Prueba 1: cómo se ve una prueba que falla

Es un ejemplo para ver la salida de un error: calcula `2 - 2` y espera `4`. Falla a propósito.

test_1.py

```python
def test_ejemplo_basico():
   resultado = 2 - 2
   assert resultado == 4, "Prueba de la suma de 2 + 2"
```

Salida de python -m pytest

```
=================================== FAILURES ===================================
_____________________________ test_ejemplo_basico ______________________________

    def test_ejemplo_basico():
       resultado = 2 - 2
>      assert resultado == 4, "Prueba de la suma de 2 + 2"
E      AssertionError: Prueba de la suma de 2 + 2
E      assert 0 == 4

test_1.py:3: AssertionError
============================ short test summary info ============================
FAILED test_1.py::test_ejemplo_basico - AssertionError: Prueba de la suma de ...
============================== 1 failed in 0.01s ===============================
```

Cómo leer la falla

- La línea con `>` es el `assert` que falló.
- `E assert 0 == 4`: el resultado fue 0 y se esperaba 4.
- Al final, `1 failed`. Para que pase habría que hacer `2 + 2`, que es lo que dice el mensaje.